

NCERT Solutions For Class 12 Computer Science

Chapter 3 – Stack

1. State TRUE or FALSE for the following cases:

- (a) Stack is a linear data structure**
- (b) Stack does not follow LIFO rule**
- (c) PUSH operation may result into underflow condition**
- (d) In POSTFIX notation for expression, operators are placed after operands**

Answer:

- (a) True
- (b) False
- (c) False
- (d) True

2. Find the output of the following code:

(a)

```
result = 0  
numberList = [10, 20, 30]  
numberList.append(40)
```

```
result = result + numberList.pop()
result = result + numberList.pop()
print("Result =", result)
```

Answer: Result = 70

(b)

```
answer = []
output = ""
answer.append('T')
answer.append('A')
answer.append('M')
ch = answer.pop()
output = output + ch
ch = answer.pop()
output = output + ch
ch = answer.pop()
output = output + ch
print("Result =", output)
```

Answer: Result = MAT

3. Write a program to reverse a string using stack.

Answer:

```
def push(stack, item):
    stack.append(item)
```

```
def pop(stack):
    if stack == []:
        return
    return stack.pop()
def reverse(string):
    n = len(string)
    stack = []
    for i in range(n):
        push(stack, string[i])
    string = ""
    for i in range(n):
        string += pop(stack)
    return string
string = input("Enter a string: ")
print("String:", string)
reversedStr = reverse(string)
print("Reversed String:", reversedStr)
```

4. For the arithmetic expression: $((2+3)*(4/2))+2$ show the parentheses-matching process step by step using a stack.

Answer:

Scan from left to right. For each opening parenthesis '(', push onto the stack. For each closing parenthesis ')', pop from the stack.

If stack is empty after entire scan and every ')' matched an '(', then the parentheses are balanced.

5. Evaluate the following postfix expressions (given $A=3$, $B=5$, $C=1$, $D=4$; show stack after each step):

(a) $AB + C *$

$A = 3, B = 5, C = 1$

Step-by-step:

Push 3 $\rightarrow [3]$

Push 5 $\rightarrow [3, 5]$

Pop 5, 3, add $\rightarrow 8$

Push 8 $\rightarrow [8]$

Push 1 $\rightarrow [8, 1]$

Pop 1, 8, multiply $\rightarrow 8$

Final Stack: $[8]$, Answer = 8

(b) $AB * C / D *$

$A = 3, B = 5, C = 1, D = 4$

Step-by-step:

Push 3 $\rightarrow [3]$

Push 5 $\rightarrow [3, 5]$

Pop 5, 3, multiply $\rightarrow 15$

Push 15 $\rightarrow [15]$

Push 1 $\rightarrow [15,1]$

Pop 1, 15, divide $\rightarrow 15/1 = 15$

Push 15 $\rightarrow [15]$

Push 4 $\rightarrow [15,4]$

Pop 4, 15, multiply $\rightarrow 60$

Final Stack: $[60]$, Answer = 60

6. Convert the following infix expressions to postfix notation, showing stack and string at each step:

(a) $A + B - C * D$

Answer: $AB+CD*-$

(b) $A * ((C + D) / E)$

Answer: $ACD+E/*$

7. Write a program to create a Stack for storing only odd numbers out of all the numbers entered by the user. Display the content of the Stack along with the largest odd number in the Stack. (Hint: Pop all elements, maintain the largest so far.)

Answer:

```
def push(stack, item):
    stack.append(item)
def pop(stack):
    if stack == []:
```

```
    return
    return stack.pop()
def oddStack(num):
    if num % 2 == 1:
        push(stack, num)
def GetLargest(stack):
    elem = pop(stack)
    large = elem
    while elem != None:
        if large < elem:
            large = elem
        elem = pop(stack)
    return large
n = int(input("How many numbers?"))
stack = []
for i in range(n):
    number = int(input("Enter number:"))
    oddStack(number)
print("Stack created is", stack)
bigNum = GetLargest(stack)
print("Largest number in stack", bigNum)
```